

LAPORAN PRAKTIKUM
STRUKTUR DATA
“IMPLEMENTASI DOUBLE LINKED LIST DALAM BAHASA
PEMROGRAMAN JAVA”

disusun oleh

Hafizh Habibullah

2511531002

Dosen Pengampu:

Dr. Wahyudi, S.T., M.T.

Asisten Praktikum:

MUHAMMAD GALID AVERO



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kita panjatkan ke hadirat Allah SWT. karena atas rahmat dan karunia-Nya laporan Praktikum Struktur Data ini dapat diselesaikan dengan baik.

Laporan ini disusun untuk memenuhi tugas praktikum pada mata kuliah Struktur Data dengan topik Implementasi Double Linked List dalam Bahasa Pemrograman Java. Melalui laporan ini, penulis berusaha menjelaskan hasil praktikum yang meliputi penjelasan mengenai kode program, tujuan, serta cara kerja program.

Penulis sangat mengharapkan kritik dan saran yang membangun agar dapat menjadi bahan perbaikan di kemudian hari. Semoga laporan ini dapat memberikan manfaat bagi penulis maupun pembaca.

Padang, 14 Mei 2026

Hafizh Habibullah

DAFTAR ISI

KATA PENGANTAR.....	1
DAFTAR ISI.....	2
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang.....	3
1.2 Tujuan.....	3
1.3 Manfaat.....	3
BAB 2 PEMBAHASAN.....	5
2.1 Implementasi Node pada Double Linked List.....	5
2.2 Implementasi Penambahan Data pada Double Linked List.....	6
2.2.1 Menambahkan Node di Depan.....	6
2.2.2 Menambahkan Node di Akhir.....	6
2.2.3 Menambahkan Node di Posisi Tertentu.....	7
2.2.4 Menampilkan Linked List.....	8
2.2.5 Method Main dan Hasil Output.....	8
2.3 Implementasi Penelusuran Data pada Double Linked List.....	9
2.3.1 Mencari Data Secara Maju (Forward Traversal).....	9
2.3.2 Mencari Data Secara Mundur (Backward Traversal).....	10
2.3.3 Method Main dan Hasil Output.....	11
2.4 Implementasi Penghapusan Data pada Double Linked List.....	12
2.4.1 Menghapus Node Bagian Awal.....	12
2.4.2 Menghapus Node Bagian Terakhir.....	12
2.4.3 Menghapus Node di Posisi Tertentu.....	13
2.4.4 Menampilkan Linked List.....	13
2.4.5 Method Main dan Hasil Output.....	14
2.5 Latihan Lagu dan Musik.....	15
BAB III KESIMPULAN.....	22
3.1 Kesimpulan.....	22
DAFTAR PUSTAKA	23

BAB I

PENDAHULUAN

1.1 Latar Belakang

Double Linked List merupakan struktur data linear yang terdiri dari sekumpulan node, dimana setiap node memiliki tiga bagian yaitu data, pointer ke node berikutnya (*next*), dan pointer ke node sebelumnya (*prev*). Berbeda dengan *Single Linked List* yang hanya dapat diakses dalam satu arah, *Double Linked List* memungkinkan proses penelusuran data dilakukan dari depan ke belakang maupun dari belakang ke depan.

Penggunaan *double linked list* memberikan fleksibilitas lebih dalam pengolahan data, terutama pada proses penyisipan dan penghapusan node karena setiap node saling terhubung dua arah. Struktur data ini banyak digunakan pada berbagai implementasi program seperti navigasi halaman, playlist musik, sistem undo-redo, dan manajemen data dinamis lainnya.

Pada praktikum ini dilakukan pembuatan program menggunakan konsep *double linked list* untuk memahami cara kerja node ganda, proses penambahan data, penghapusan data, serta penelusuran data secara maju dan mundur. Melalui praktikum ini diharapkan pemahaman mengenai struktur data *double linked list* dapat meningkat serta mampu diterapkan dalam pembuatan program yang lebih kompleks.

1.2 Tujuan

Adapun tujuan dari praktikum ini adalah:

1. Memahami konsep dasar struktur data Double Linked List.
2. Memahami hubungan antar node menggunakan pointer *next* dan *prev*.
3. Mengimplementasikan *Double linked list* ke dalam program Java.
4. Mampu melakukan operasi penambahan, penghapusan, dan penelusuran data pada *Double Linked List*.

1.3 Manfaat

Manfaat dari Praktikum ini adalah:

1. Menambah pemahaman mengenai struktur data linear khususnya *Double Linked List*.
2. Membantu memahami proses manipulasi data secara dua arah dalam program.
3. Melatih kemampuan pemrograman Java dalam pembuatan struktur data dinamis.
4. Mempermudah pemahaman konsep node, pointer, dan hubungan antar data.
5. Menjadi dasar dalam pengembangan program yang membutuhkan navigasi data dua arah.

BAB 2

PEMBAHASAN

2.1 Implementasi Node pada Double Linked List

```
1 package pekan6_2511531002;
2
3 public class NodeDLL_2511531002 {
4     int data_1002;
5     NodeDLL_2511531002 next_1002;
6     NodeDLL_2511531002 prev_1002;
7
8     public NodeDLL_2511531002(int data_1002) {
9         this.data_1002 = data_1002;
10        this.next_1002 = null;
11        this.prev_1002 = null;
12    }
13 }
```

Kode Program 2. 1 – Program Node pada Double Linked List

Class **NodeDLL_2511531002** merupakan class yang digunakan sebagai representasi node pada struktur data *Double Linked List*. Class ini memiliki atribut **data_1002** yang bertipe integer untuk menyimpan nilai data, atribut **next_1002** sebagai pointer yang menunjuk ke node berikutnya, serta atribut **prev_1002** sebagai pointer yang menunjuk ke node sebelumnya.

Pada class ini juga terdapat constructor **NodeDLL_2511531002(int data_1002)** yang berfungsi untuk menginisialisasi data node saat objek dibuat. Nilai parameter yang diterima akan disimpan ke atribut **data_1002**, sedangkan pointer **next_1002** dan **prev_1002** diatur bernilai null karena node belum terhubung dengan node lain.

Class ini menjadi dasar utama dalam implementasi *Double Linked List* karena setiap data yang disimpan di dalam list direpresentasikan dalam bentuk node yang saling terhubung dua arah.

2.2 Implementasi Penambahan Data pada Double Linked List

2.2.1 Menambahkan Node di Depan

```
1 package pekan6_2511531002;
2
3 public class InsertDLL_2511531002 {
4     static NodeDLL_2511531002 insertBegin_1002(NodeDLL_2511531002 head_1002, int data_1002) {
5         NodeDLL_2511531002 new_node_1002 = new NodeDLL_2511531002(data_1002);
6         new_node_1002.next_1002 = head_1002;
7         if (head_1002 != null) {
8             head_1002.prev_1002 = new_node_1002;
9         }
10        return new_node_1002;
11    }
}
```

Kode Program 2.2 – Blok Program Menambahkan Node di Depan

Method ini digunakan untuk menambahkan node baru pada bagian awal *Double Linked List*. Method ini menerima parameter **head_1002** sebagai node awal list dan **data_1002** sebagai data yang akan dimasukkan.

Pada method ini dibuat node baru menggunakan constructor **NodeDLL_2511531002**. Selanjutnya pointer **next_1002** dari node baru diarahkan ke node head sebelumnya. Jika list tidak kosong, maka pointer **prev_1002** dari head lama akan diarahkan kembali ke node baru. Setelah proses selesai, node baru dikembalikan sebagai head terbaru dari *Double Linked List*.

2.2.2 Menambahkan Node di Akhir

```
public static NodeDLL_2511531002 insertEnd_1002(NodeDLL_2511531002 head_1002, int newData_1002) {
    NodeDLL_2511531002 newNode_1002 = new NodeDLL_2511531002(newData_1002);
    if (head_1002 == null) {
        head_1002 = newNode_1002;
    } else {
        NodeDLL_2511531002 curr_1002 = head_1002;
        while (curr_1002.next_1002 != null) {
            curr_1002 = curr_1002.next_1002;
        }
        curr_1002.next_1002 = newNode_1002;
        newNode_1002.prev_1002 = curr_1002;
    }
    return head_1002;
}
```

Kode Program 2.3 – Blok Program Menambahkan Node di Akhir

Method ini digunakan untuk menambahkan node baru pada bagian akhir *Double Linked List*. Method ini menerima parameter **head_1002** sebagai head list dan **newData_1002** sebagai data yang akan ditambahkan.

Proses dimulai dengan membuat node baru. Jika list masih kosong, maka node baru langsung dijadikan head. Namun jika list sudah memiliki isi, program akan melakukan traversal menggunakan variabel **curr_1002**

hingga mencapai node terakhir. Setelah node terakhir ditemukan, pointer **next_1002** node terakhir diarahkan ke node baru dan pointer **prev_1002** node baru diarahkan kembali ke node sebelumnya. Method kemudian mengembalikan head list.

2.2.3 Menambahkan Node di Posisi Tertentu

```
public static NodeDLL_2511531002 insertAtPosition_1002(NodeDLL_2511531002 head_1002, int pos_1002, int new_data_1002) {
    NodeDLL_2511531002 new_node_1002 = new NodeDLL_2511531002(new_data_1002);
    if (pos_1002 == 1) {
        new_node_1002.next_1002 = head_1002;
        if (head_1002 != null) {
            head_1002.prev_1002 = new_node_1002;
        }
        head_1002 = new_node_1002;
        return head_1002;
    }
    NodeDLL_2511531002 curr_1002 = head_1002;
    for (int i = 1; i < pos_1002 - 1 && curr_1002 != null; ++i) {
        curr_1002 = curr_1002.next_1002;
    }
    if (curr_1002 == null) {
        System.out.println("Posisi tidak ada");
        return head_1002;
    }
    new_node_1002.prev_1002 = curr_1002;
    new_node_1002.next_1002 = curr_1002.next_1002;
    curr_1002.next_1002 = new_node_1002;
    if (new_node_1002.next_1002 != null) {
        new_node_1002.next_1002.prev_1002 = new_node_1002;
    }
    return head_1002;
}
```

Kode Program 2.4 – Blok Program Menambahkan Node di Posisi Tertentu

Method ini digunakan untuk menambahkan node baru pada posisi tertentu dalam *Double Linked List*. Method ini menerima parameter **head_1002**, **pos_1002** sebagai posisi penyisipan, **new_data_1002** sebagai data baru.

Jika posisi yang dimasukkan adalah posisi pertama, maka node baru akan langsung ditempatkan di awal list dengan cara yang sama seperti method **insertBegin_1002()**. Jika posisi bukan pertama, program akan melakukan traversal hingga mencapai node sebelum posisi tujuan menggunakan perulangan **for**.

Setelah posisi ditemukan, pointer **prev_1002** dan **next_1002** pada node baru akan diatur agar terhubung dengan node sebelum dan sesudahnya. Jika posisi tidak ditemukan karena melebihi panjang list, program akan menampilkan pesan “Posisi tidak ada”.

2.2.4 Menampilkan Linked List

```
public static void printList_1002(NodeDLL_2511531002 head_1002) {
    NodeDLL_2511531002 curr_1002 = head_1002;
    while (curr_1002 != null) {
        System.out.print(curr_1002.data_1002 + " ↔ ");
        curr_1002 = curr_1002.next_1002;
    }
    System.out.println();
}
```

Kode Program 2.5 – Blok Program Menampilkan Linked List

Method ini digunakan untuk menampilkan seluruh isi *Double Linked List*. Method ini menerima parameter **head_1002** sebagai node awal list. Program melakukan traversal dari head hingga node terakhir menggunakan pointer **next_1002**. Setiap data node akan dicetak ke layar dengan format <-> sebagai penanda hubungan antar node. Method ini membantu pengguna melihat isi list setelah dilakukan operasi penambahan data.

2.2.5 Method Main dan Hasil Output

```
64 public static void main(String[] args) {
65     // membuat dll 2 ↔ 3 ↔ 5
66     NodeDLL_2511531002 head_1002 = new NodeDLL_2511531002(2);
67     head_1002.next_1002 = new NodeDLL_2511531002(3);
68     head_1002.next_1002.prev_1002 = head_1002;
69     head_1002.next_1002.next_1002 = new NodeDLL_2511531002(5);
70     head_1002.next_1002.next_1002.prev_1002 = head_1002.next_1002;
71
72     // cetak dll awal
73     System.out.print("DLL Awal: ");
74     printList_1002(head_1002);
75
76     // tambah 1 di awal
77     head_1002 = insertBegin_1002(head_1002, 1);
78     System.out.print("Simpul 1 ditambah di awal: ");
79     printList_1002(head_1002);
80
81     // tambah 6 di akhir
82     System.out.print("Simpul 6 ditambah di akhir: ");
83     int data_1002 = 6;
84     head_1002 = insertEnd_1002(head_1002, data_1002);
85     printList_1002(head_1002);
86
87     // menambah node 4 di posisi 4
88     System.out.print("Tambah node 4 di posisi 4: ");
89     int data2_1002 = 4;
90     int pos_1002 = 4;
91     head_1002 = insertAtPosition_1002(head_1002, pos_1002, data2_1002);
92     printList_1002(head_1002);
93 }
94 }
```

Kode Program 2.6 – Blok Program Main

Method ini merupakan method utama yang digunakan untuk menjalankan program. Pada method ini dibuat *Double Linked List* awal dengan data 2 <-> 3 <-> 5. Setelah itu program memanggil beberapa method untuk melakukan operasi pada list.

Program pertama kali mencetak isi list awal menggunakan method **printList_1002()**. Selanjutnya dilakukan penambahan node bernilai 1 di awal list menggunakan method **insertBegin_1002()**. Setelah itu node bernilai 6 ditambahkan di akhir list menggunakan method **insertEnd_1002()**. Program juga menambahkan node bernilai 4 pada posisi ke-4 menggunakan method **insertAtPosition_1002()**.

Setiap perubahan pada *Double Linked List* akan ditampilkan ke layar agar hasil operasi dapat dilihat secara langsung. Hasil output dari program ini adalah sebagai berikut:



```
<terminated> InsertDLL_2511531002 [Java Application] C:\Users\user\p2\pool\plugins\
DLL Awal: 2 <-> 3 <-> 5 <->
Simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
Simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
Tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->
```

Gambar 2.1 – Hasil Program Penambahan Data pada Double Linked List

2.3 Implementasi Penelusuran Data pada Double Linked List

2.3.1 Mencari Data Secara Maju (Forward Traversal)

```
1 package pekan6_2511531002;
2
3 public class PenelusuranDLL_2511531002 {
4     static void forwardTraversal_1002(NodeDLL_2511531002 head_1002) {
5         NodeDLL_2511531002 curr_1002 = head_1002;
6         while (curr_1002 != null) {
7             System.out.print(curr_1002.data_1002 + " <-> ");
8             curr_1002 = curr_1002.next_1002;
9         }
10        System.out.println();
11    }
```

Kode Program 2.7 – Blok Program Traversal Maju

Method ini digunakan untuk melakukan penelusuran data secara maju pada *Double Linked List*. Method ini menerima parameter **head_1002** sebagai node awal list.

Proses traversal dilakukan menggunakan variabel **curr_1002** yang awalnya menunjuk ke head. Selama node tidak bernilai **null**, data pada setiap node akan ditampilkan ke layar dengan format <->. Setelah itu pointer akan berpindah ke node berikutnya menggunakan **next_1002** hingga mencapai node terakhir.

Method ini menunjukkan cara kerja *Double Linked List* dalam melakukan penelusuran dari depan ke belakang.

2.3.2 Mencari Data Secara Mundur (Backward Traversal)

```
static void backwardTraversal_1002(NodeDLL_2511531002 tail_1002) {  
    NodeDLL_2511531002 curr_1002 = tail_1002;  
    while (curr_1002 != null) {  
        System.out.print(curr_1002.data_1002 + " ↔ ");  
        curr_1002 = curr_1002.prev_1002;  
    }  
    System.out.println();  
}
```

Kode Program 2.8 – Blok Program Traversal Mundur

Method ini digunakan untuk melakukan penelusuran data secara mundur pada *Double Linked List*. Method ini menerima parameter **tail_1002** sebagai node terakhir list.

Traversal dilakukan menggunakan variabel **curr_1002** yang dimulai dari node tail. Data setiap node akan ditampilkan ke layar selama node tidak bernilai **null**. Perpindahan node dilakukan menggunakan pointer **prev_1002** sehingga penelusuran berlangsung dari belakang ke depan.

Method ini membuktikan bahwa *Double Linked List* dapat diakses dua arah karena setiap node memiliki pointer ke node sebelumnya dan node berikutnya.

2.3.3 Method Main dan Hasil Output

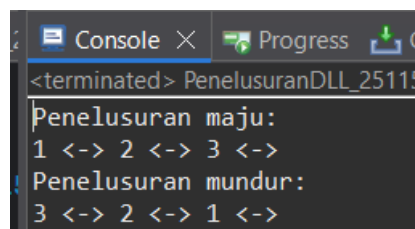
```
22● public static void main(String[] args) {  
23     NodeDLL_2511531002 head_1002 = new NodeDLL_2511531002(1);  
24     NodeDLL_2511531002 second_1002 = new NodeDLL_2511531002(2);  
25     NodeDLL_2511531002 third_1002 = new NodeDLL_2511531002(3);  
26  
27     head_1002.next_1002 = second_1002;  
28     second_1002.prev_1002 = head_1002;  
29     second_1002.next_1002 = third_1002;  
30     third_1002.prev_1002 = second_1002;  
31  
32     System.out.println("Penelusuran maju: ");  
33     forwardTraversal_1002(head_1002);  
34  
35     System.out.println("Penelusuran mundur: ");  
36     backwardTraversal_1002(third_1002);  
37 }  
38 }
```

Kode Program 2.9 – Blok Program Main

Method ini merupakan method utama yang digunakan untuk menjalankan program penelusuran *Double Linked List*. Pada method ini dibuat tiga node dengan data 1, 2, dan 3. Selanjutnya setiap node dihubungkan menggunakan pointer **next_1002** dan **prev_1002** sehingga membentuk struktur *Double Linked List*.

Program kemudian memanggil method **forwardTraversal_1002()** untuk menampilkan isi list secara maju mulai dari head. Setelah itu method **backwardTraversal_1002()** dipanggil untuk menampilkan isi list secara mundur mulai dari tail.

Hasil dari program ini menunjukkan bahwa *Double Linked List* dapat ditelusuri dari dua arah, yaitu dari depan ke belakang maupun dari belakang ke depan. Hasil output programnya adalah sebagai berikut:



```
Console X Progress G  
<terminated> PenelusuranDLL_25115  
Penelusuran maju:  
1 <-> 2 <-> 3 <->  
Penelusuran mundur:  
3 <-> 2 <-> 1 <->
```

Gambar 2. 2 – Hasil Program Pencarian Data pada SLL

2.4 Implementasi Penghapusan Data pada Double Linked List

2.4.1 Menghapus Node Bagian Awal

```
1 package pekan6_2511531002;  
2  
3 public class HapusDLL_2511531002 {  
4     public static NodeDLL_2511531002 delHead_1002(NodeDLL_2511531002 head_1002) {  
5         if (head_1002 == null) return null;  
6         head_1002 = head_1002.next_1002;  
7         if (head_1002 != null) head_1002.prev_1002 = null;  
8         return head_1002;  
9     }  
10 }
```

Kode Program 2.10 – Blok Program Menghapus Bagian Awal

Method ini digunakan untuk menghapus node pada bagian awal *Double Linked List*. Method ini menerima parameter **head_1002** sebagai node awal list.

Program terlebih dahulu memeriksa apakah list kosong. Jika kosong, method akan mengembalikan nilai **null**. Jika list memiliki isi, maka head akan dipindahkan ke node berikutnya menggunakan pointer **next_1002**. Setelah itu pointer **prev_1002** pada head baru diatur menjadi **null** agar tidak lagi terhubung dengan node lama yang telah dihapus. Method kemudian mengembalikan head terbaru dari *Double Linked List*.

2.4.2 Menghapus Node Bagian Terakhir

```
11 public static NodeDLL_2511531002 delLast_1002(NodeDLL_2511531002 head_1002) {  
12     if (head_1002 == null) return null;  
13     if (head_1002.next_1002 == null) return null;  
14     NodeDLL_2511531002 curr_1002 = head_1002;  
15     while (curr_1002.next_1002 != null) curr_1002 = curr_1002.next_1002;  
16     if (curr_1002.prev_1002 != null) curr_1002.prev_1002.next_1002 = null;  
17     return head_1002;  
18 }
```

Kode Program 2.11 – Blok Program Menghapus Node Terakhir

Method ini digunakan untuk menghapus node terakhir pada *Double Linked List*. Method ini menerima parameter **head_1002** sebagai node awal list.

Program akan memeriksa apakah list kosong atau hanya memiliki satu node. Jika kondisi tersebut terjadi, method akan mengembalikan **null**. Jika list memiliki lebih dari satu node, program melakukan traversal menggunakan variabel **curr_1002** hingga mencapai node terakhir.

Setelah node terakhir ditemukan, pointer **next_1002** dari node sebelumnya akan diatur menjadi **null** sehingga hubungan dengan node terakhir terputus. Method kemudian mengembalikan head list.

2.4.3 Menghapus Node di Posisi Tertentu

```
public static NodeDLL_2511531002 delPos_1002(NodeDLL_2511531002 head_1002, int pos_1002) {
    if (head_1002 == null) return head_1002;
    NodeDLL_2511531002 curr_1002 = head_1002;
    for (int i = 1; curr_1002 != null && i < pos_1002; ++i ) curr_1002 = curr_1002.next_1002;
    if (curr_1002 == null) return head_1002;
    if (curr_1002.prev_1002 != null) curr_1002.prev_1002.next_1002 = curr_1002.next_1002;
    if (curr_1002.next_1002 != null) curr_1002.next_1002.prev_1002 = curr_1002.prev_1002;
    if (head_1002 == curr_1002) head_1002 = curr_1002.next_1002;
    return head_1002;
}
```

Kode Program 2.12 – Blok Program Menghapus Node di Posisi Tertentu

Method ini digunakan untuk menghapus node pada posisi tertentu dalam *Double Linked List*. Method ini menerima parameter **head_1002** dan **pos_1002** sebagai posisi node yang akan dihapus. Program akan melakukan traversal menggunakan variabel **curr_1002** hingga mencapai posisi yang ditentukan. Jika posisi tidak ditemukan, method langsung mengembalikan head tanpa perubahan.

Jika node ditemukan, maka pointer **next_1002** dari node sebelumnya akan diarahkan ke node sesudahnya, sedangkan pointer **prev_1002** dari node sesudahnya akan diarahkan ke node sebelumnya. Jika node yang dihapus merupakan head, maka head akan dipindahkan ke node berikutnya. Method ini memastikan hubungan antar node tetap terhubung dengan benar setelah proses penghapusan dilakukan.

2.4.4 Menampilkan Linked List

```
public static void printList_1002(NodeDLL_2511531002 head_1002) {
    NodeDLL_2511531002 curr_1002 = head_1002;
    while (curr_1002 != null) {
        System.out.print(curr_1002.data_1002 + " ");
        curr_1002 = curr_1002.next_1002;
    }
    System.out.println();
}
```

Kode Program 2.13 – Blok Program Menampilkan Linked List

Method ini digunakan untuk menampilkan isi *Double Linked List*. Method ini menerima parameter **head_1002** sebagai node awal list.

Program melakukan traversal dari head hingga node terakhir menggunakan pointer **next_1002**. Setiap data node akan dicetak ke layar sehingga pengguna dapat melihat isi list setelah dilakukan operasi penghapusan node.

2.4.5 Method Main dan Hasil Output

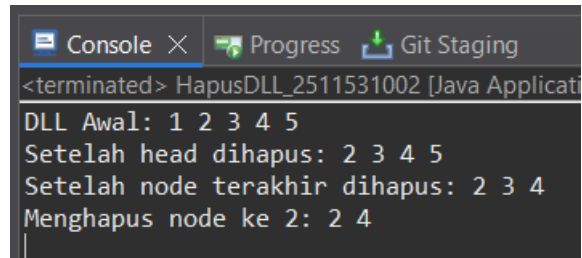
```
40 public static void main(String[] args) {
41     NodeDLL_2511531002 head_1002 = new NodeDLL_2511531002(1);
42     head_1002.next_1002 = new NodeDLL_2511531002(2);
43     head_1002.next_1002.prev_1002 = head_1002;
44
45     head_1002.next_1002.next_1002 = new NodeDLL_2511531002(3);
46     head_1002.next_1002.next_1002.prev_1002 = head_1002.next_1002;
47
48     head_1002.next_1002.next_1002.next_1002 = new NodeDLL_2511531002(4);
49     head_1002.next_1002.next_1002.next_1002.prev_1002 = head_1002.next_1002.next_1002;
50
51     head_1002.next_1002.next_1002.next_1002.next_1002 = new NodeDLL_2511531002(5);
52     head_1002.next_1002.next_1002.next_1002.next_1002.prev_1002 = head_1002.next_1002.next_1002.next_1002;
53
54     System.out.print("DLL Awal: ");
55     printList_1002(head_1002);
56
57     System.out.print("Setelah head dihapus: ");
58     head_1002 = delHead_1002(head_1002);
59     printList_1002(head_1002);
60
61     System.out.print("Setelah node terakhir dihapus: ");
62     head_1002 = delLast_1002(head_1002);
63     printList_1002(head_1002);
64
65     System.out.print("Menghapus node ke 2: ");
66     head_1002 = delPos_1002(head_1002, 2);
67     printList_1002(head_1002);
68 }
69 }
```

Kode Program 2.14 – Blok Program Main

Method ini merupakan method utama yang digunakan untuk menjalankan program penghapusan node pada *Double Linked List*. Pada method ini dibuat list dengan data **1 2 3 4 5** yang saling terhubung menggunakan pointer **next_1002** dan **prev_1002**.

Program pertama kali menampilkan isi list awal menggunakan method **printList_1002()**. Selanjutnya dilakukan penghapusan node pertama menggunakan method **delHead_1002()**, kemudian penghapusan node terakhir menggunakan method **delLast_1002()**, dan terakhir penghapusan node pada posisi kedua menggunakan method **delPos_1002()**.

Setiap hasil operasi penghapusan akan ditampilkan ke layar agar perubahan pada *Double Linked List* dapat dilihat secara langsung. Hasil Output dari program adalah sebagai berikut:



```
<terminated> HapusDLL_2511531002 [Java Applicati
DLL Awal: 1 2 3 4 5
Setelah head dihapus: 2 3 4 5
Setelah node terakhir dihapus: 2 3 4
Menghapus node ke 2: 2 4
```

Gambar 2. 3 – Hasil Program Penghapusan Node pada DLL

2.5 Latihan Lagu dan Musik

```
package pekan6_2511531002;

public class Lagu_2511531002 {
    private String judul_1002;
    private String penyanyi_1002;
    Lagu_2511531002 next_1002;
    Lagu_2511531002 prev_1002;

    // Constructor
    public Lagu_2511531002(String judul_1002, String
    penyanyi_1002) {
        this.judul_1002 = judul_1002;
        this.penyanyi_1002 = penyanyi_1002;
        this.next_1002 = null;
        this.prev_1002 = null;
    }

    // Getter
    public String getJudul_1002() {
        return judul_1002;
    }
    public String getPenyanyi_1002() {
        return penyanyi_1002;
    }

    // Setter
    public void setJudul_1002(String judul_1002) {
        this.judul_1002 = judul_1002;
    }
    public void setPenyanyi_1002(String penyanyi_1002) {
        this.penyanyi_1002 = penyanyi_1002;
    }
}
```

Kode Program 2.15 – Tugas Pasien_2511531002

Class ini digunakan untuk menyimpan data lagu pada program playlist menggunakan struktur data *Double Linked List*. Pada class ini terdapat atribut **judul_1002** untuk menyimpan judul lagu dan **penyanyi_1002** untuk menyimpan

nama penyanyi. Selain itu, terdapat juga atribut **next_1002** dan **prev_1002** yang berfungsi untuk menghubungkan node dengan node sesudah dan sebelumnya.

Constructor pada class ini digunakan untuk memasukkan data lagu saat objek dibuat. Ketika sebuah objek lagu dibuat, judul dan penyanyi akan langsung diisi sesuai input, sedangkan **next_1002** dan **prev_1002** bernilai **null** karena node belum terhubung ke node lain.

Class ini juga memiliki getter dan setter yang digunakan untuk mengambil serta mengubah data judul dan penyanyi. Dengan adanya class ini, data lagu dapat disimpan dan dihubungkan satu sama lain sehingga playlist dapat ditampilkan dari depan maupun dari belakang.

```
package pekan6_2511531002;
import java.util.Scanner;

public class Musik_2511531002 {
    Lagu_2511531002 head_1002;
    Lagu_2511531002 tail_1002;

    // INSERT
    public void tambahLagu_1002(String judul_1002, String
    penyanyi_1002) {
        Lagu_2511531002 laguBaru_1002 = new
        Lagu_2511531002(judul_1002, penyanyi_1002);

        if (head_1002 == null) { //Jika playlist kosong
            head_1002 = laguBaru_1002;
            tail_1002 = laguBaru_1002;
        } else {
            tail_1002.next_1002 = laguBaru_1002;
            laguBaru_1002.prev_1002 = tail_1002;
            tail_1002 = laguBaru_1002;
        }

        System.out.println("Lagu berhasil ditambahkan!");
    }

    // DELETE HEAD
    public void hapusLaguAwal_1002() {
        if (head_1002 == null) { // Jika playlist kosong
            System.out.println("Playlist kosong!");
            return;
        }

        System.out.println("Lagu \"" +
        head_1002.getJudul_1002() + "\" berhasil dihapus.");

        if (head_1002 == tail_1002) { // Jika hanya ada satu
        lagu
```

```

        head_1002 = null;
        tail_1002 = null;
    } else {
        head_1002 = head_1002.next_1002;
        head_1002.prev_1002 = null;
    }
}

// DISPLAY FORWARD
public void tampilMaju_1002() {
    if (head_1002 == null) { // Jika playlist kosong
        System.out.println("Playlist kosong!");
        return;
    }

    Lagu_2511531002 bantu_1002 = head_1002;

    System.out.println("\n=== Playlist Maju ===");
    while (bantu_1002 != null) {
        System.out.println("Judul      : " +
bantu_1002.getJudul_1002());
        System.out.println("Penyanyi : " +
bantu_1002.getPenyanyi_1002());
        System.out.println("-----");
    };

    bantu_1002 = bantu_1002.next_1002;
}

// DISPLAY BACKWARD
public void tampilMundur_1002() {
    if (tail_1002 == null) { // Jika playlist kosong
        System.out.println("Playlist kosong!");
        return;
    }

    Lagu_2511531002 bantu_1002 = tail_1002;

    System.out.println("\n=== Playlist Mundur ===");
    while (bantu_1002 != null) {
        System.out.println("Judul      : " +
bantu_1002.getJudul_1002());
        System.out.println("Penyanyi : " +
bantu_1002.getPenyanyi_1002());
        System.out.println("-----");
    };

    bantu_1002 = bantu_1002.prev_1002;
}

// SEARCH
public void cariLagu_1002(String judulCari_1002) {
    if (head_1002 == null) { // Jika playlist kosong
        System.out.println("Playlist kosong!");
        return;
    }
}

```

```

        Lagu_2511531002 bantu_1002 = head_1002;
        boolean ketemu_1002 = false;

        while (bantu_1002 != null) {
            if
(bantu_1002.getJudul_1002().equalsIgnoreCase(judulCari_1002)) {
                System.out.println("\nLagu ditemukan!");
                System.out.println("Judul      : " +
bantu_1002.getJudul_1002());
                System.out.println("Penyanyi : " +
bantu_1002.getPenyanyi_1002());

                ketemu_1002 = true;
                break;
            }
            bantu_1002 = bantu_1002.next_1002;
        }
        if (!ketemu_1002) { // Jika lagu tidak ketemu
            System.out.println("Lagu tidak ditemukan!");
        }
    }

    // MAIN
    public static void main(String[] args) {
        Scanner input_1002 = new Scanner(System.in);
        Musik_2511531002 playlist_1002 = new
Musik_2511531002();
        int pilihan_1002;

        do {
            System.out.println("\n=== Playlist Musik NIM:
2511531002 ===");
            System.out.println("1. Tambah Lagu");
            System.out.println("2. Hapus Lagu Pertama");
            System.out.println("3. Lihat Playlist (Maju)");
            System.out.println("4. Lihat Playlist
(Mundur)");

            System.out.println("5. Cari Lagu");
            System.out.println("6. Keluar");
            System.out.print("Pilihan: ");

            pilihan_1002 = input_1002.nextInt();
            input_1002.nextLine();

            switch (pilihan_1002) {
                case 1:
                    System.out.print("Judul      : ");
                    String judul_1002 =
input_1002.nextLine();

                    System.out.print("Penyanyi : ");
                    String penyanyi_1002 =
input_1002.nextLine();

                    playlist_1002.tambahLagu_1002(judul_1002,
penyanyi_1002);
                    break;
            }
        } while (true);
    }
}

```

```

        case 2:
            playlist_1002.hapusLaguAwal_1002();
            break;
        case 3:
            playlist_1002.tampilMaju_1002();
            break;
        case 4:
            playlist_1002.tampilMundur_1002();
            break;
        case 5:
            System.out.print("Masukkan judul lagu
yang dicari : ");
            String cari_1002 = input_1002.nextLine();

            playlist_1002.cariLagu_1002(cari_1002);
            break;
        case 6:
            System.out.println("Program selesai...");
            break;
        default:
            System.out.println("Pilihan tidak
valid!");
    }
    } while (pilihan_1002 != 6);

    input_1002.close();
}
}

```

Kode Program 2.16 – Tugas RumahSakit_2511531002

Program **Musik_2511531002** merupakan program utama yang digunakan untuk mengelola playlist lagu menggunakan struktur data *Double Linked List*. Program ini memanfaatkan class **Lagu_2511531002** sebagai node untuk menyimpan data lagu. Variabel **head_1002** digunakan sebagai penunjuk lagu pertama, sedangkan **tail_1002** digunakan sebagai penunjuk lagu terakhir pada playlist.

Program memiliki beberapa method utama seperti **tambahLagu_1002()** untuk menambahkan lagu ke playlist, **hapusLaguAwal_1002()** untuk menghapus lagu pertama, **tampilMaju_1002()** untuk menampilkan playlist dari depan ke belakang, **tampilMundur_1002()** untuk menampilkan playlist dari belakang ke depan, serta **cariLagu_1002()** untuk mencari lagu berdasarkan judul.

Pada method **main()**, program berjalan menggunakan perulangan **do-while** yang akan terus menampilkan menu hingga pengguna memilih keluar. Menu yang tersedia meliputi penambahan lagu, penghapusan lagu pertama, menampilkan

playlist maju dan mundur, serta pencarian lagu. Struktur **switch-case** digunakan untuk menjalankan method sesuai pilihan pengguna. Program akan berhenti ketika pengguna memilih opsi keluar, kemudian Scanner ditutup. Hasil output program akan terlihat seperti berikut:

```
Console X Progress Git Staging
<terminated> Musik_2511531002 [Java Application] C:\Users\u

=== Playlist Musik NIM: 2511531002 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul    : Best Friend
Penyanyi : Rex Orange County
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511531002 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)|
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul    : EARTHQUAKE
Penyanyi : Tyler, The Creator
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511531002 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul    : Too Little, Too Late
Penyanyi : Laufey my beloved
Lagu berhasil ditambahkan!
```

```
Console X Progress Git Staging
<terminated> Musik_2511531002 [Java Application] C:\User

=== Playlist Musik NIM: 2511531002 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu "Best Friend" berhasil dihapus.
```

```
Console X Progress Git Staging
<terminated> Musik_2511531002 [Java Application] C:\Use

=== Playlist Musik NIM: 2511531002 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3

=== Playlist Maju ===
Judul   : EARFQUAKE
Penyanyi : Tyler, The Creator
-----
Judul   : Too Little, Too Late
Penyanyi : Laufey my beloved
-----

=== Playlist Musik NIM: 2511531002 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4

=== Playlist Mundur ===
Judul   : Too Little, Too Late
Penyanyi : Laufey my beloved
-----
Judul   : EARFQUAKE
Penyanyi : Tyler, The Creator
-----
```

```
=== Playlist Musik NIM: 2511531002 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu yang dicari : earfquake

Lagu ditemukan!
Judul   : EARFQUAKE
Penyanyi : Tyler, The Creator

=== Playlist Musik NIM: 2511531002 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 6
Program selesai...
```

Gambar 2. 4 – Hasil Program Musik_2511531002

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data *Double Linked List* memungkinkan data diakses dan dikelola dalam dua arah menggunakan pointer **next** dan **prev**. Melalui implementasi program, operasi seperti penambahan node, penghapusan node, pencarian data, serta penelusuran maju dan mundur dapat dilakukan dengan baik.

Pada praktikum ini juga dilakukan penugasan membuat program playlist musik menggunakan *Double Linked List*. Program tersebut mampu menambahkan lagu, menghapus lagu pertama, menampilkan playlist dari dua arah, serta mencari lagu berdasarkan judul. Dari hasil praktikum dapat dipahami bahwa *Double Linked List* lebih fleksibel dibanding *Single Linked List* karena setiap node saling terhubung ke node sebelumnya dan berikutnya.

Dengan adanya praktikum ini, pemahaman mengenai konsep node, pointer, dan manipulasi data pada struktur data *Double Linked List* menjadi lebih baik serta dapat diterapkan dalam pembuatan program yang lebih kompleks.

DAFTAR PUSTAKA

- [1] Modul Praktikum Struktur Data Pekan 6 – Double Linked List.